MARST Notebook — Plain-English Walkthrough

A guided tour of `marst-multigpu.ipynb` for someone who has never coded before. We start with what the notebook is trying to do, the vocabulary you need, and then walk through every cell in order.

---

1. What is this notebook actually doing?

Imagine 325 sensors planted along the freeways of the San Francisco Bay Area, each one reporting the speed of passing cars every 5 minutes. Now imagine that 80% of those sensors went dark — broken hardware, network outage, whatever. We're left with a sparse pattern of speeds.

The question: can a computer fill in the missing speeds, just by looking at the few sensors that *are* reporting and at how speeds normally behave at that time of day?

That's called **imputation**. This notebook builds a model called **MARST** ("Multi-Anchor Residual Spatiotemporal Transformer") that solves this problem, compares it against 18 other methods, runs it on 4 different traffic datasets, and produces a stack of figures showing that MARST wins.

The key word **streaming** (or **causal**) means: when filling in time `t`, the model is only allowed to look at past times `t-1, t-2, ...`. No peeking at the future. This is what makes the problem honest — in real life the future hasn't happened yet.

---

2. Vocabulary you'll need

Before we open the code, here are the words that show up everywhere:

- **Tensor** — a multi-dimensional spreadsheet of numbers. A `[T, N]` tensor is `T` rows by `N` columns. A `[B, N, T]` tensor is a stack of `B` such spreadsheets. PyTorch (the library) is basically a calculator for tensors.
- **GPU** (Graphics Processing Unit) — a chip that does millions of small calculations in parallel. Training a model on a CPU takes hours; on a GPU, minutes. Kaggle gives you 2 GPUs to play with.
- **Model** — a function with millions of tunable knobs (called **parameters** or **weights**). At first the knobs are random and the function is useless. **Training** is the process of slowly adjusting the knobs so the function gets good at the job.
- **Loss** — a number that measures "how wrong is the model right now?" Training is the act of repeatedly nudging the knobs to make the loss smaller.
- **Epoch** — one full pass of training. 800 epochs means we nudged the knobs 800 times worth of work.
- **Seed** — a random starting point. Two training runs with the same seed produce identical results; different seeds produce slightly different models. Reporting "mean over 3 seeds" is more honest than reporting one lucky run.
- **MAE** (Mean Absolute Error) — the average of `|guess - truth|`. Smaller is better. Our headline number.
- **Mask** — a tensor of 1s and 0s, same shape as the data. 1 means "this sensor is observed (visible)" and 0 means "this sensor is hidden (we have to guess it)."
- **Sparsity** — the fraction of 0s in the mask. 80% sparsity means 80% of sensors are hidden.

- **Anchor** — a cheap, hand-coded guess at the answer. MARST uses three anchors (LOCF, HA, KNN — explained later) and learns to blend them.
- **Notebook cell** — a paragraph of code or text in a Jupyter notebook. Cells are run one at a time. Variables created in one cell are visible in all later cells.

---

3. The three anchors MARST blends

The clever idea of MARST is that we already know three dumb-but-decent ways to guess a missing sensor reading. The model doesn't have to figure these out from scratch — they're given to it as inputs:

1. **LOCF** = "Last Observation Carried Forward." If sensor 7 last reported 58 mph and now it's dark, just guess 58 mph again. Good when speeds change slowly.
2. **HA** = "Historical Average." Look at what sensor 7 normally reads at 5pm on a weekday and guess that. Good when speeds follow a daily pattern.
3. **KNN** = "K-Nearest Neighbors." Look at the neighbouring sensors that *are* reporting and average them. Good when nearby sensors agree.

MARST is a neural network that, for each (sensor, time) pair, decides via a softmax which of the three anchors to trust the most, then adds a small correction. That's the "Multi-Anchor Residual" in its name.

---

4. So what makes MARST novel?

Reasonable question to ask at this point: if LOCF, HA, and KNN are all textbook methods from decades ago, what's actually new here? The honest answer is that the *anchors themselves* aren't novel — any classical time-series book has them — but **what MARST does with them** is the contribution. Five things to point at:

4.1 The anchors are inputs, not predictions

Classical work uses LOCF or HA *as* the answer. MARST uses them as **features fed into a transformer**. So instead of asking the network to discover from scratch that "speeds are temporally persistent," we hand it the LOCF guess and let it decide where that guess is trustworthy. The network spends its capacity on *correcting* simple priors instead of *rediscovering* them.

4.2 The blend is learned and per-position

The output isn't `α·LOCF + β·HA + γ·KNN` with fixed weights — that would be trivial. It's:

```
π(sensor, time) = softmax(transformer_output)        # 3 numbers, sum to 1
blended         = π[LOCF]·LOCF + π[HA]·HA + π[KNN]·KNN
prediction      = blended + α · residual
```

The mixture weights `π` change for **every (sensor, time) pair**. So MARST can say "at 5pm on a Tuesday at sensor 7, trust LOCF 0.61 / HA 0.33 / KNN 0.06" and at the same instant for sensor 12 say "trust LOCF 0.22 / HA 0.74 / KNN 0.04."

The result tables confirm this is exactly what happens — PEMS-BAY learns to lean on LOCF (because speeds change slowly), PEMS04/08 learn to lean on HA (because flow has strong daily structure). No prior work combines those three causal anchors with a learned per-position gate and a residual correction.

4.3 Residual learning on top of a sensible prior

Even with the gate set to uniform, the blended anchor is already a decent guess. The transformer only has to learn a small *correction* (`α · residual`, where `α` starts at 0.05). Compare to BRITS or iTransformer: they're learning the entire prediction from raw inputs. MARST is learning the **delta from a known-good baseline**. This is the same idea that made ResNet work in computer vision — it's easier to learn small corrections than full functions from scratch.

4.4 Strict causality + anti-leak invariants

A lot of imputation literature (SAITS, bidirectional BRITS) lets the model peek at future timesteps when filling in time `t`. That's fine for offline imputation but useless for live traffic systems. MARST is **strictly causal** — at time `t` it only reads times `≤ t`, and the leak audit at the end of every dataset run proves it (`max|dpast| = 0.00e+00 → CAUSAL`).

Three concrete anti-leak invariants are enforced: - **Zero diagonal on the adjacency matrix**, so a sensor can never read its own value as a "neighbour." - **Causal soft-LOCF**, so the running LOCF state at time `t` only consults observed values from times `≤ t`. - **500-step train/eval gap**, so the LOCF anchor in eval can't be carrying training values forward.

Not unique to MARST individually, but the package is enforced *and audited* — every dataset run prints `AUDIT PASSED: no model reads held-out data; ST model is strictly causal.`

4.5 Engineering pieces that aren't anchors but matter

These don't show up in the name "MARST" but contribute to the wins:

- **Jam-weighted Huber loss.** Congested positions get 2× weight during training. Most papers train uniformly; we explicitly optimise for the regime that actually matters for traffic. Hard to find this exact recipe in prior imputation literature.
- **Sparsity curriculum** (60% → 80% over the first 75% of epochs). The ablation table in `REPORT.md` shows this is worth +0.07 to +1.60 MAE depending on dataset — it's not cosmetic.
- **Learnable per-sensor embeddings.** At 80% sparsity a sensor is mostly invisible — its input features are mostly zeros. The 128-dimensional embedding tells the transformer "this is sensor 7 with these typical characteristics" even when the value is masked. Without it the model can't tell hidden sensors apart from each other.
- **Soft-LOCF with decay.** A smoothly-decaying LOCF that drifts toward HA over time, instead of LOCF abruptly returning to nothing. The decay sweep confirms `d=0.95` is materially better than `0.80`, `0.90`, or `0.99` on every dataset.

So what would the paper claim, in one sentence?

> A causal spatiotemporal transformer that imputes by **learning a per-position softmax mixture over three interpretable temporal/spatial priors** (LOCF, HA, KNN), refined by a small residual, trained with a sparsity curriculum and a jam-weighted loss.

The novelty isn't a new attention mechanism or a new graph layer — those parts are standard. The novelty is the **decomposition strategy**: factor the prediction into "interpretable cheap guesses + learned gating + small residual" and prove the gate is doing real work (Wilcoxon $p<0.001$ vs every baseline, and the anchor mix π shifts sensibly across datasets).

Where the claim is weakest

Two reviewer objections to be ready for:

1. **"You're just adding domain-specific feature engineering."** Partly true — providing LOCF/HA/KNN as inputs *is* feature engineering. The defence: the network learns *when* to trust each anchor, which is the contribution, and the alternative (giving the network only raw values and hoping it discovers these regularities) is empirically worse — that's what the 18 baselines without explicit anchors are showing.
2. **"Your anchor ablation on speed datasets isn't clean."** Also partly true — on METR-LA, removing the HA anchor *helps* by 0.18 mph; on PEMS-BAY, removing KNN helps by 0.04. The honest framing: the multi-anchor gate is **clearly load-bearing on flow datasets** (PEMS04 and PEMS08, where every leave-one-out hurts by +1 to +3) and **roughly neutral on speed** (METR-LA and PEMS-BAY at this compute budget). Don't oversell.

One-sentence explanation for a non-expert

> "LOCF, HA, and KNN are old tricks that each work in some situations and fail in others. MARST is a neural network that looks at each missing sensor reading and decides, on the spot, which mix of those tricks to trust — and then fixes the result a little. It's the *learned, per-position blending* that's new, not the tricks themselves."

That's a defensible claim and one you can back with the JamMAE table, the anchor-mixture ablation, and the Wilcoxon test in `REPORT.md`.

---

5. Has anyone published something similar?

Reasonable follow-up: "if the strategy is so sensible, surely someone has done it before?" Sort of yes, sort of no. The *strategy* (combine a classical predictor with a neural network and let the network learn corrections) has a long pedigree. The *specific instantiation* MARST uses (per-position softmax mixture over LOCF/HA/KNN inside a strictly-causal spatiotemporal transformer for traffic imputation) does not appear in the published literature. Here's the landscape.

5.1 The strategy is well-established — three Tier-1 precedents

- **ES-RNN (Smyl, 2020)** — won the **M4 forecasting competition** by a wide margin over 60+ submissions. Combined classical **Exponential Smoothing** with an LSTM: ES handles trend and seasonality (the "structured" part), LSTM learns deviations. This is exactly MARST's strategy applied to forecasting instead of imputation.
- **Wide & Deep (Cheng et al., Google, 2016)** — combined a **wide linear model** (interpretable memorisation) with a **deep neural network** (generalisation). Productionised at Google Play with over 1 billion users. Same complementarity argument: classical and neural carry different kinds of capacity.

- **N-BEATS (Oreshkin et al., 2020, ICLR)** — interpretable basis decomposition with polynomial (trend) and Fourier (seasonality) bases. Same decomposition philosophy as MARST, but the priors are mathematical basis functions instead of operational anchors.

So when a reviewer says "you're just using textbook methods," the honest reply is: **using textbook methods this way is a published, recognised paradigm.** Three high-impact papers establish it. MARST applies it in a place no one has applied it before.

5.2 The closest cousins in spatiotemporal imputation

- **STAMImputer (IJCAI 2025)** — Mixture-of-Experts for traffic imputation with softmax gating. **Closest design.** But its experts are neural modules (multi-head attention, graph attention, FFN), not classical anchors. Also bidirectional, not causal. And tested on different traffic datasets (PemsD8, taxi data) — not the standard PEMS-BAY/METR-LA benchmarks.
- **ImputeFormer (KDD 2024)** — current published SOTA on PEMS-BAY/METR-LA. Transformer with low-rank attention constraints. No anchors, no gating. MARST's main competitor on headline numbers.
- **GRIN (Cini et al., ICLR 2022)** — recurrent + message-passing graph neural network. Major baseline. Reduces MAE by ~29% over BRITS on PEMS-BAY.
- **SAITS, BRITS** — already in our benchmark (#11 and #12); MARST beats both significantly.

5.3 What the 2024 imputation survey confirms

A 2024 survey of 11+ state-of-the-art methods (arXiv 2412.04733) explicitly confirms:

- **None** of the surveyed methods feed LOCF / HA / KNN as input features.
- **None** use a learned per-position softmax mixture over multiple imputation priors.

That's the gap MARST fills. It's not an architectural novelty in the "we invented a new attention mechanism" sense — it's a **decomposition strategy** novelty: factor the prediction into "interpretable cheap guesses + learned per-position gating + small residual" and demonstrate empirically that it works.

5.4 Positioning matrix

Method	Year	Causal?	Classical priors as inputs?	Per-position mixture?	PEMS-BAY/METR-LA tested?
BRITS	2018	bidir	no	no	yes
GRIN	2022	bidir	no	no	yes
SAITS	2022	no	no	no	–
ImputeFormer	2024	no	no	no	yes
STAMImputer	2025	bidir	no	yes (neural experts)	no
Bridge-TS	2025	no	neural priors	no (diffusion refine)	–
MARST (ours)	–	**yes**	**yes (LOCF+HA+KNN)**	**yes (per sensor, time)**	**yes (all 4)**

That last row is the gap.

The full version of this discussion (with quotes, references, BibTeX entries, and detailed citations) lives in `RELATED_WORK.md`.

---

6. Cell-by-cell walkthrough

Cell 0 — title and abstract (markdown)

A markdown cell, meaning it's prose, not code. It introduces the notebook: 4 datasets, 80% sparsity protocol, the multi-GPU strategy. Nothing runs here.

Cell 1 — related work (markdown)

More prose: situates this work in the imputation literature. Streaming/causal vs offline, what other methods do. No code.

Cell 2 — clear the GPU memory

```
import torch
import gc
gc.collect()
torch.cuda.empty_cache()
print("GPU memory ready.")
```

Five lines that do a single thing: free up any leftover GPU memory before we start. `gc.collect()` asks Python to take out the garbage. `torch.cuda.empty_cache()` tells the GPU to release memory it was holding. The `print` confirms we got here.

You only need this when re-running the notebook in the same kernel session. Fresh starts don't need it, but it's defensive.

Cell 3 — imports, configuration, dataset metadata

This is a long cell that sets up everything global. Let's break it into parts.

3a. Import the libraries we'll use:

```
import torch                    # the tensor calculator
import torch.nn as nn           # neural network building blocks
import torch.nn.functional as F # activation functions, loss functions
import numpy as np              # tensor calculator for CPU
import pandas as pd             # for reading .csv / .h5 spreadsheet files
import os, glob, pickle         # file system, file globbing, save/load Python objects
import urllib.request           # for downloading the datasets
import warnings, json           # suppress noise, save results
from concurrent.futures import ThreadPoolExecutor  # for running many training jobs in
parallel
```

Each `import` is "open the toolbox and let me use these tools." Nothing runs yet — we're just collecting tools.

3b. Suppress warning chatter:

```
warnings.filterwarnings("ignore")
```

PyTorch occasionally prints yellow "deprecated, may go away in v3.0" warnings. We mute them so the output stays readable.

3c. Pin the global random seed:

```
GLOBAL_SEED = 42
torch.manual_seed(GLOBAL_SEED)
np.random.seed(GLOBAL_SEED)
```

Random operations (initial model weights, shuffling, etc.) all consult a pseudo-random sequence. Seeding with 42 means "start that sequence at point 42" — and that point is the same on every run, so the experiment is reproducible.

3d. Detect the GPU(s):

```
device  = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
N_GPUS  = torch.cuda.device_count() if torch.cuda.is_available() else 1
GPU_IDS = list(range(N_GPUS))
print(f'Multi-GPU: detected {N_GPUS} CUDA device(s) -> concurrent per-GPU jobs across
{GPU_IDS}')
```

- `device` is where tensors will live. Prefer GPU; fall back to CPU if no GPU is plugged in.
- `N_GPUS` is the count of GPUs (1 on most machines, 2 on Kaggle T4×2).
- `GPU_IDS` is the list `[0, 1]` we'll use to round-robin work across both GPUs.

3e. Experiment hyperparameters:

```
SPARSITY        = 0.80      # fraction of sensors hidden at any moment
BATCH_TIME      = 48        # each training window is 48 time-steps = 4 hours
HIDDEN_DIM      = 96        # internal width of the transformer (knobs per layer)
N_LAYERS        = 5         # how many transformer blocks stacked
N_HEADS         = 4         # multi-head attention heads
DROPOUT         = 0.1       # randomly zero 10% of activations during training
(regularisation)
TRAIN_EPOCHS    = 800       # main training duration
ABLATION_EPOCHS = 400       # shorter budget for ablation experiments
N_BASELINE_SEEDS = 3        # train each baseline 3 times for noise estimates
JAM_WEIGHT       = 1.0      # multiply loss by 2x on jam (congested) positions
STEPS_PER_DAY = 288         # 24h * 60min / 5min = 288 timesteps/day
EVAL_SEEDS    = [42, 43, 44, 45, 46]   # 5 different random eval masks
HUBER_BETA    = 1.0         # Huber loss transition point (less sensitive to outliers)
```

These are the dials you'd turn if you wanted to try a different configuration. They're set once at the top so it's obvious what the experiment is.

3f. Train/eval split:

```
WINDOW     = 5000     # use first 5000 timesteps total
TRAIN_END  = 4000     # train on steps 0-3999
EVAL_START = 4500     # evaluate on steps 4500-4949 (the 500-step gap prevents leakage)
EVAL_LEN   = 450
```

We hold out the last chunk of time for testing. The 500-step gap is important: if train ended at 4000 and eval started at 4001, the LOCF anchor would carry training values straight into eval — a form of cheating.

3g. Dataset metadata:

```
DATASETS = {
    'PEMS-BAY': dict(kind='speed', steps_per_day=288, unit='mph',
                     data_files=['pems-bay.h5', ...],
                     adj_files=['adj_mx_bay.pkl', ...],
                     data_url='https://zenodo.org/.../PEMS-BAY.csv?download=1',
                     adj_url='https://zenodo.org/.../adj_mx_bay.pkl?download=1'),
    'METR-LA': dict(...),
    'PEMS04':  dict(...),
    'PEMS08':  dict(...),
}
```

A dictionary that maps a dataset name to its details: what kind of measurements it has (speed or flow), what unit (mph or veh/5min), what filename(s) to look for on disk, and what URL to download from if missing. This is just a lookup table — running it creates the dictionary but doesn't load any data.

3h. File-finding helpers:

```
DATA_DIR = None

def _roots():
    cands = [DATA_DIR, '.', os.getcwd(), '/content', '/content/drive/MyDrive',
             '/kaggle/input', '/kaggle/working']
    out = []
    for r in cands:
        if r and os.path.isdir(r) and r not in out:
            out.append(r)
    return out
```

`_roots` returns a list of folders to search for dataset files. It checks the current folder, Google Colab's `/content`, Kaggle's `/kaggle/input`, and a few others. We try each in turn until we find what we need.

```
def find_file(candidates, search_roots=None):
    if search_roots is None:
        search_roots = _roots()
    cand_lower = [c.lower() for c in candidates]
    for root in search_roots:
        if not os.path.isdir(root):
            continue
        for path in glob.glob(os.path.join(root, '**', '*'), recursive=True):
            if os.path.isfile(path) and os.path.basename(path).lower() in cand_lower:
                return path
    return None
```

`find_file` walks every folder recursively, asking: "is there a file here whose name (lowercased) matches one of the names I'm looking for?" Returns the first match, or `None`.

```
def download_if_missing(url, dest):
    if url is None:
        return None
    if not os.path.exists(dest):
        print(f"Downloading {dest} from {url} ...")
        urllib.request.urlretrieve(url, dest)
    return dest
```

If the file isn't there, fetch it from the URL. Simple safety net so the notebook is self-contained.

3i. Data loaders:

```
def load_raw_array():
    """Return value_raw [T, N] float32: speed (km/h) or flow (veh/5min)."""
    if CFG['kind'] == 'speed':
        h5 = find_file([f for f in CFG['data_files'] if f.lower().endswith('.h5')])
        if h5 is not None:
            df = pd.read_hdf(h5)
            return np.nan_to_num(df.values.astype(np.float32), nan=0.0)
        ...
    else:  # flow
        npz = find_file([f for f in CFG['data_files'] if f.lower().endswith('.npz')])
        ...
```

`load_raw_array` returns a 2D tensor: rows = timesteps, columns = sensors. Speed datasets store .h5 or .csv; flow datasets store .npz. `np.nan_to_num` replaces "not a number" placeholders with 0.

```
def load_adjacency(num_nodes):
    """Return binary adjacency [N, N], diagonal zeroed."""
    if CFG['kind'] == 'speed':
        pkl = find_file(CFG['adj_files'])
        with open(pkl, 'rb') as f:
            obj = pickle.load(f, encoding='latin1')
        adj_mx = obj[2] if isinstance(obj, (list, tuple)) and len(obj) >= 3 else obj
        ...
    np.fill_diagonal(adj, 0)
    return adj
```

`load_adjacency` returns an N×N matrix where `adj[i][j] = 1` if sensor `i` and sensor `j` are road-network neighbours. The diagonal is zeroed because we never want a sensor to "read its own value" as a neighbour — that would be the same cheating problem in space instead of time.

3j. Print confirmation:

```
print(f"Device: {device} | Datasets: {list(DATASETS)} | Target Sparsity:
{SPARSITY*100:.0f}%")
```

Just a sanity-check line so you can confirm the config got set up correctly.

Cell 4 — STBlock (the transformer building block)

A `class` in Python is a custom data type. Here we define `STBlock` ("Spatiotemporal Block") — one layer of the MARST model.

```
class STBlock(nn.Module):
    def __init__(self, hidden, n_heads, ff_mult=2, dropout=0.1):
        super().__init__()
        ...
```

Inheriting from `nn.Module` makes this a recognized PyTorch model component (it can be trained, saved, etc.). `__init__` is what runs when you create one. It defines all the sub-layers:

- A self-attention layer (for time)
- A self-attention layer (for space)
- A feed-forward network
- Layer normalisations and dropouts to keep training stable

```
    def forward(self, x):
        # time-axis attention
        ...
        # space-axis attention
        ...
        # feed-forward
        ...
        return x
```

`forward` is what happens when you actually pass data through. It takes a tensor `x` of shape `[batch, nodes, time, hidden]`, runs it through the attention layers, and returns a tensor of the same shape but with more useful internal representations.

You can think of `STBlock` as one slice of the model. MARST stacks 6 of them.

Cells 5–12 — section headers (markdown)

Just markdown landmarks: "Models," "Recent Causal Baselines," "MARST," "Leak Audit," "Sparsity Sensitivity," etc. They make the notebook navigable but run no code.

Cell 13 — `run_dataset(name)`: the entire pipeline

This is the giant function (≈1700 lines) that does the actual work for one dataset. It's called four times (once per dataset) from cell 15.

```
def run_dataset(name):
    global ANCHOR_ABLATION_EPOCHS, A_t, BATCH_SIZE_MARST, ...
```

The `global` line is a Python technicality: variables we assign inside the function should also be visible outside it (for the later analysis cells). The long list registers all of them.

We'll walk through `run_dataset` in 17 sub-sections.

13.1 — Pick the dataset

```
print(f"#  RUN DATASET: {DATASET}  ({CFG['kind']}, {int(SPARSITY*100)}% sparsity)")
DATASET = name
CFG = DATASETS[DATASET]
```

Grab the metadata for the chosen dataset (e.g. `name='PEMS-BAY'` → `CFG` becomes the PEMS-BAY entry from the big dictionary).

13.2 — Load the raw data

```
value_raw = load_raw_array()   # [T, N] e.g. [5000, 325] for PEMS-BAY
if CFG['kind'] == 'speed':
    valid_raw = (value_raw > 0).astype(np.float32)
else:
    valid_raw = np.ones_like(value_raw, dtype=np.float32)
```

`value_raw` is the speeds (or flows). For speed datasets, the convention is that "0" means "broken sensor" — so we build a `valid_raw` tensor marking which entries are real. For flow datasets, every entry is real, so `valid_raw` is all 1s.

```
print(f"  Valid fraction: {valid_raw.mean():.3f}")
```

Confirms what fraction of the data is real. For PEMS-BAY ~1.0 (all good); for METR-LA ~0.94 (6% genuinely missing).

13.3 — Clamp range and regime edges

```
CLAMP_LO = 0.0
CLAMP_HI = max(value_raw.max(), 120.0)
REGIME_EDGES = (np.percentile(_vt, 20), np.percentile(_vt, 40))
```

`CLAMP_LO/HI` are sanity bounds — speeds < 0 or > a freeway max are physically impossible. `REGIME_EDGES` define the boundary between "jam" (slowest 20%) and "free-flow" — used later to compute JamMAE.

13.4 — Compute per-sensor normalization

```
node_means = ...   # average value per sensor over training data only
node_stds  = ...   # standard deviation per sensor over training data only
speed_norm = (value_raw - node_means) / node_stds   # z-score
```

Neural networks learn faster if inputs are roughly mean-zero, std-one. We compute each sensor's mean and standard deviation using only the training window (no peeking at the test window) and rescale all values.

13.5 — Compute the Historical Average prior

```
ha_prior = np.zeros_like(speed_norm)
for s in range(STEPS_PER_DAY):
    sel = slot_idx[:TRAIN_END] == s
    sub_data  = speed_norm[:TRAIN_END][sel]
    sub_valid = valid_raw [:TRAIN_END][sel]
    sums = (sub_data * sub_valid).sum(axis=0)
    cnts = sub_valid.sum(axis=0) + 1e-8
    ha_prior[slot_idx == s] = sums / cnts
```

For each 5-minute slot of the day (0–287), look at every observation of every sensor that ever happened in that slot during training, average them, and fill that into `ha_prior` at every position with the same slot. The result: `ha_prior[t][n]` answers "what does sensor `n` typically read at time-of-day `t`?"

This becomes the **HA anchor** later.

13.6 — Move tensors to the GPU

```
speed_gpu = torch.tensor(speed_norm).to(device)
valid_gpu = torch.tensor(valid_raw).to(device)
ha_prior  = torch.tensor(ha_prior ).to(device)
```

Numpy lives on the CPU. PyTorch can use the GPU. `.to(device)` ships the array over to GPU memory so subsequent operations are fast.

13.7 — Build the evaluation masks

```
def make_eval_mask_np(seed, EL, N, sparsity=None):
    rng = np.random.default_rng(seed)
    sparsity = SPARSITY if sparsity is None else sparsity
    mask = (rng.random((EL, N)) > sparsity).astype(np.float32)
    return mask
```

For each of the 5 `EVAL_SEEDS`, we generate an `[EL, N]` random mask where each cell is 1 with probability `1 - sparsity = 20%`. Five different seeds → five different test problems, so we can report mean and std.

```
_max_jac = max pairwise Jaccard of the 5 masks
assert _max_jac < SPARSITY/(2-SPARSITY) + 0.10
```

A sanity check: if two of our "different" masks happened to overlap too much, the 5 test scores wouldn't really be independent. We verify they don't.

13.8 — Classical baselines (#1–#5)

Five non-neural methods, used as floor references.

```
# 1. Historical Average — just predict ha_prior at every masked position
# 2. LOCF — at each timestep, carry forward the last observed value
# 3. Global Mean — predict each sensor's overall training mean
# 4. Node-wise Ridge — fit a linear model per sensor on time-of-day features
# 5. KNN Imputer — for each blind position, find similar training time-windows
#                  by Euclidean distance on observed sensors, average them
```

Each prints its MAE per eval-seed. KNN and LOCF are the strongest classical baselines.

13.9 — Per-node MLP baseline (#6)

```
class MLP(nn.Module):
    ...
NE, NH, NLR, NBT = 800, 64, 1e-3, 48   # epochs, hidden, lr, batch_time
for seed in range(3):                   # 3 training seeds for noise estimate
    net = MLP(4, NH).to(device)
    opt = torch.optim.Adam(net.parameters(), lr=NLR)
    sch = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=NE)
    for ep in range(1, NE+1):
        net.train()
        # sample random training window
        # form features: [value*mask, mask, sin(time-of-day), cos(time-of-day)]
        # forward, compute loss on masked positions only, backward, step
```

A tiny feed-forward network for each sensor. The features are very simple — observed value, mask, and a 2D time-of-day encoding (`sin` and `cos` make times like "23:55" and "00:05" appear close, which they are).

The 3 lines after the inner loop are the classical training step: - `opt.zero_grad()` — wipe last step's gradients - `loss.backward()` — compute new gradients - `opt.step()` — nudge the knobs

This pattern repeats for every neural baseline.

13.10 — More neural baselines (#7–#12, #13–#18)

The notebook implements 13 more neural baselines:

- **#7 LSTM, #8 2L-LSTM, #9 GRU** — recurrent networks: read the time series one step at a time, maintain hidden state
- **#10 TCN** — causal dilated convolutions over time
- **#11 SAITS** — self-attention imputation transformer
- **#12 BRITS** — forward-only GRU for imputation
- **#13 DCRNN** — diffusion-convolutional RNN (uses the adjacency)
- **#14 GWN** — Graph WaveNet (adaptive graph + gated TCN)
- **#15 STID** — identity-style baseline with sensor + time-of-day embeddings
- **#16 DLinear** — decompose into trend + seasonality, fit linear
- **#17 PatchTST** — patch-based transformer
- **#18 iTransformer** — invert the axes; attend across sensors instead of across time

Each one follows the same pattern: define a class, train for some epochs across `N_BASELINE_SEEDS=3` seeds, evaluate on the 5 eval masks, print mean ± std.

13.11 — MARST model class (the headliner)

```
class MaskedSTTransformerMARST(nn.Module):
    def __init__(self, num_nodes, adj, node_means_t, node_stds_t,
                 hidden=128, n_heads=4, n_layers=6, max_T=288, dropout=0.1,
                 soft_locf_decay=0.95):
        super().__init__()
        self.register_buffer('adj_static', adj)
        self.register_buffer('node_means', node_means_t)
        self.register_buffer('node_stds',  node_stds_t)
        self.register_buffer('adj_2hop', ...)
        # learnable layers:
        self.in_proj    = nn.Linear(9, hidden)   # 9 input features per (sensor, time)
        self.node_emb   = nn.Parameter(torch.randn(num_nodes, hidden) * 0.02)
        self.blocks     = nn.ModuleList([STBlock(...) for _ in range(n_layers)])
        self.anchor_gate = nn.Linear(hidden, 3)  # picks which anchor to trust
        self.alpha       = nn.Parameter(torch.tensor(0.05))  # residual scale
        self.out_head    = nn.Linear(hidden, 1)
```

- `register_buffer` stores tensors that are *part of the model state* but not trained (adjacency, normalisation statistics).
- `nn.Parameter` and `nn.Linear` create trainable knobs.
- `node_emb` gives each sensor its own learnable 128-d signature — crucial because at 80% sparsity, a sensor might be invisible most of the time; the embedding tells the model "this is sensor 7" even when the value is masked.

Then the forward pass:

```
def forward(self, x, m, t_sin, t_cos, ha_prior):
    B, N, T = x.shape
    mean_v = self.node_means.view(1, -1, 1)
    std_v  = self.node_stds .view(1, -1, 1)

    locf, staleness, soft_locf = self._compute_causal_signals(x, m, ha_prior)
    locf_kmh = locf * std_v + mean_v
    # 3 anchors:
    a_locf = locf
    a_ha   = ha_prior
    a_knn  = self._knn_anchor(x, m, ha_prior)
    n_mean_2hop = (torch.matmul(self.adj_2hop, locf_kmh) - mean_v) / std_v

    # Build 9-feature input: value, mask, time encoding, 3 anchors, staleness, neighbour
mean
    feat = torch.stack([x, m, t_sin, t_cos, a_locf, a_ha, a_knn,
                        staleness, n_mean_2hop], dim=-1)
    h = self.in_proj(feat)
    h = h + self.node_emb.view(1, N, 1, self.hidden)   # add sensor identity

    for blk in self.blocks:
        h = blk(h)   # 6 STBlock layers stacked

    pi      = torch.softmax(self.anchor_gate(h), dim=-1)   # 3 weights summing to 1
    anchors = torch.stack([a_locf, a_ha, a_knn], dim=-1)
    blended = (pi * anchors).sum(dim=-1)                    # weighted blend
    self.last_pi = pi.detach().reshape(-1, 3)               # for diagnostics

    residual = self.out_head(h).squeeze(-1) * self.alpha
    return blended + residual
```

In plain English: build a 9-channel "feature picture" at every (sensor, time) point (observed value, mask, time-of-day, 3 anchor guesses, how stale the LOCF anchor is, 2-hop neighbour mean). Run it through 6 transformer layers. Use the final representation to (a) decide weights `π` over the 3 anchors and (b) add a small correction `α · residual`. The output is the blended anchor plus the correction.

The `_compute_causal_signals` helper builds the LOCF and "soft-LOCF" anchors by walking through time and remembering the most recent observed value per sensor. The `_knn_anchor` averages over observed road neighbours.

13.12 — MARST ablation class

```
class MaskedSTTransformerMARSTAblate(nn.Module):
    def __init__(self, ..., use_locf=True, use_ha=True, use_knn=True):
        ...
```

A variant that lets you turn off individual anchors at construction time. Used in the anchor-mixture ablation to see how much each anchor matters.

13.13 — Shared training helpers

```
TRAIN_SEEDS = [0, 1, 2]

def _train_st(ctor, epochs, batch_size, train_seed, label,
              use_curriculum=True, gpu_id=0):
    dev = torch.device(f'cuda:{gpu_id}') if torch.cuda.is_available() else
torch.device('cpu')
    if torch.cuda.is_available():
        torch.cuda.set_device(dev)
    sg, vg, hp = speed_gpu.to(dev), valid_gpu.to(dev), ha_prior.to(dev)
    nm_t, ns_t = node_means_t.to(dev), node_stds_t.to(dev)
    rng_np = np.random.default_rng(train_seed)
    g_cuda = torch.Generator(device=dev)
    g_cuda.manual_seed(int(train_seed) * 31 + int(gpu_id))
    torch.manual_seed(train_seed)
    net = ctor().to(dev)
    opt = torch.optim.Adam(net.parameters(), lr=1e-3)
    sch = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
    curr_end = max(1, int(epochs * 0.75))
    for ep in range(1, epochs + 1):
        net.train()
        if use_curriculum:
            sparsity_ep = min(SPARSITY, 0.60 + (SPARSITY - 0.60) *
                              min(1.0, (ep - 1) / curr_end))
        else:
            sparsity_ep = SPARSITY
        t0_list = rng_np.integers(0, TRAIN_END - BATCH_TIME, batch_size)
        # build a [B, N, T] training window from random time-offsets ...
        # forward, jam-weighted Huber loss, backward, step, clip grads
    if torch.cuda.is_available():
        net = net.to(torch.device("cuda:0"))
        torch.cuda.set_device(torch.device("cuda:0"))
    return net
```

This is the key function. Pieces:

- `gpu_id` says which GPU this run should use. Multiple of these can run at once on different GPUs.
- **Per-thread copies** of the data tensors (`sg, vg, hp, nm_t, ns_t`) live on the assigned GPU.
- `rng_np` **and** `g_cuda` are thread-local random sources so concurrent workers don't step on each other's RNG state.
- **Curriculum:** for the first 75% of epochs, gradually ramp sparsity from 60% to the target 80%. Starting easier helps the model learn. After epoch `curr_end`, sparsity stays at 80%.
- **Inside the loop:** pick `batch_size` random time offsets, slice out `[batch_size, N, BATCH_TIME]` windows, generate fresh per-batch random masks, forward, loss, backward, step.
- **Jam-weighted loss:** congested positions get 2× weight, so the model pays more attention to jams (which are what really matter for traffic prediction).
- **At the end** move the trained model back to GPU 0 explicitly so downstream eval code finds it there.

```
def _train_parallel(jobs):
    if N_GPUS <= 1:
        return [_train_st(...) for j in jobs]
    with ThreadPoolExecutor(max_workers=N_GPUS) as ex:
        futs = [ex.submit(_train_st, ..., gpu_id=i % N_GPUS) for i, j in enumerate(jobs)]
        return [f.result() for f in futs]
```

Dispatches a list of training jobs across the GPUs. With 2 GPUs and 3 seeds: seed 0 starts on GPU 0, seed 1 on GPU 1, both train simultaneously; once a slot frees, seed 2 starts. Total wall time ≈ 2 sequential runs instead of 3.

```
def _eval_st_mae(net):
    net.eval()
    # for each of 5 EVAL_SEEDS, build a mask, forward, compute MAE
    # return shape [5] of per-seed MAEs
```

After training, evaluate on the 5 fixed eval-mask seeds. Returns the array of MAEs that we'll summarise.

13.14 — Train MARST main (3 seeds, in parallel)

```
HIDDEN_DIM_MARST = 128
N_LAYERS_MARST   = 6
TRAIN_EPOCHS_MARST = 800
BATCH_SIZE_MARST = 4
ctor_marst = lambda: MaskedSTTransformerMARST(NUM_NODES, A_t, ...).to(device)

marst_mask_mat = []
_marst_jobs = [dict(ctor=ctor_marst, epochs=TRAIN_EPOCHS_MARST,
                    batch_size=BATCH_SIZE_MARST, seed=tseed,
                    label=f"MARST(seed={tseed})") for tseed in TRAIN_SEEDS]
_marst_nets = _train_parallel(_marst_jobs)
for si, (tseed, net) in enumerate(zip(TRAIN_SEEDS, _marst_nets)):
    marst_mask_mat.append(_eval_st_mae(net))
    if si == 0:
        net_marst = net   # save the first one for later analysis
```

Build 3 training jobs (one per seed), dispatch them concurrently across GPUs, then evaluate. The seed-0 model is kept as `net_marst` for figures and the extended-metrics table.

13.15 — Extended metrics, leak audit

The extended-metrics block computes 10 quality measures per model (MAE, JamMAE, RMSE, MedAE, MAPE%, R^2, Pearson, MBE, Hit@N, Hit@2N) using each model's saved predictions. Prints a big table per dataset.

```
# Leak audit: corrupt the held-out truth and check that predictions don't move
for each model:
    pred_clean = model(eval_input)
    eval_input_corrupted = eval_input + huge_noise_on_held_out_positions
    pred_corrupted = model(eval_input_corrupted)
    assert max|pred_clean - pred_corrupted| ≈ 0
```

If a model "secretly" reads the held-out truth, the corruption would change its predictions. Every model passes `NO LEAK`. MARST also gets a stricter `CAUSAL` audit (perturbing future inputs doesn't change past outputs).

13.16 — Sparsity sensitivity sweep

```
SPARSITY_LEVELS = [0.50, 0.80, 0.95]
for sp in SPARSITY_LEVELS:
    SPARSITY_SWEEP['MARST'][sp] = _eval_at_sparsity(net_marst, sp, 'model')
    SPARSITY_SWEEP['LOCF'][sp]  = _eval_at_sparsity(None, sp, 'locf')
    SPARSITY_SWEEP['HA'][sp]    = _eval_at_sparsity(None, sp, 'ha')
```

Re-evaluate the already-trained MARST (no retraining) at three test sparsities. Probes robustness: does the model still work when even more sensors are blind than it was trained on?

13.17 — Anchor ablation (concurrent)

```
MARST_ABLATIONS = {
    'Full MARST (LOCF+HA+KNN)': dict(),
    '- LOCF anchor':            dict(use_locf=False),
    '- HA anchor':              dict(use_ha=False),
    '- KNN anchor':             dict(use_knn=False),
    'LOCF only':                dict(use_ha=False, use_knn=False),
    'HA only':                  dict(use_locf=False, use_knn=False),
    'KNN only':                 dict(use_locf=False, use_ha=False),
}
_ab_jobs = [dict(ctor=..., epochs=ANCHOR_ABLATION_EPOCHS, ...) for variant in
MARST_ABLATIONS]
_ab_nets = _train_parallel(_ab_jobs)
```

Train 7 model variants concurrently (2 at a time on 2 GPUs), at the shorter 400-epoch budget. Each variant turns some anchors off so we can see which ones really mattered.

13.18 — Soft-LOCF decay sweep (concurrent)

```
SOFT_LOCF_DECAY_SWEEP = {0.95: marst_mask_mat[0].copy()}
_decays = [0.80, 0.90, 0.99]
_decay_jobs = [dict(ctor=lambda d=d: MaskedSTTransformerMARST(...,
soft_locf_decay=d).to(device),
                    epochs=ANCHOR_ABLATION_EPOCHS, ...) for d in _decays]
_decay_nets = _train_parallel(_decay_jobs)
```

The "soft LOCF" inside MARST has a decay parameter that controls how much weight to put on older observed values. Sweep 3 variants concurrently.

13.19 — Curriculum vs fixed

```
_net_fix = _train_st(ctor_marst, ANCHOR_ABLATION_EPOCHS, BATCH_SIZE_MARST, 0,
                     "MARST-fixed", use_curriculum=False)
```

Train one more variant of MARST with `use_curriculum=False` (sparsity stays at 80% from epoch 0) and compare against the main curriculum run. Confirms the curriculum is doing useful work.

13.20 — Missing-pattern robustness

```
MISSING_PATTERNS = ['point', 'block', 'sensor']
for pat in MISSING_PATTERNS:
    # generate masks where missingness is patterned (random points / time blocks /
    # whole sensors blind), and evaluate every trained model on each pattern
```

Real-world missingness isn't always random. A broken sensor goes dark for hours, not just one timestep. This block tests how each model copes when the missingness is structured.

13.21 — Save results to JSON

```
save_obj = dict(
    dataset=DATASET,
    kind=CFG['kind'],
    sparsity=SPARSITY,
    results={k: [float(x) for x in v] for k, v in RESULTS.items()},
    extended_metrics=EXT_METRICS,
    ablation=globals().get('ABLATION_SAVE', {}),
    sparsity_sweep=globals().get('SPARSITY_SAVE', {}),
    marst_anchor_ablation=globals().get('MARST_ANCHOR_ABLATION_SAVE', {}),
    ...
)
with open(f'results_{DATASET}.json', 'w') as f:
    json.dump(save_obj, f)
print(f"Saved results_{DATASET}.json")
```

Dump everything to a JSON file. The aggregation cells later read these to build the cross-dataset comparison.

Cell 14 — markdown header

Just text: "Cross-dataset aggregation."

Cell 15 — driver loop

```
import os, traceback
DATASETS_TO_RUN = ['PEMS-BAY', 'METR-LA', 'PEMS04', 'PEMS08']
FORCE_RERUN = False
for _ds in DATASETS_TO_RUN:
    _out = f'results_{_ds}.json'
    if os.path.exists(_out) and not FORCE_RERUN:
        print(f'skip {_ds}: {_out} already exists')
        continue
    try:
        run_dataset(_ds)
    except Exception as e:
        print(f"!! FAILED {_ds}: {e}")
        traceback.print_exc()
print("All requested datasets processed.")
```

For each of the four datasets, if `results_<name>.json` doesn't already exist, run the full pipeline. The `try/except` means: if one dataset fails (a download timeout, say), keep going with the others. `FORCE_RERUN = False` means restarted sessions resume from where they left off.

Cells 16–47 — cross-dataset analysis

These are smaller standalone cells that read the saved JSONs and produce the final tables and figures.

- **Cell 17:** the cross-dataset MAE table (rows = models, columns = datasets, MARST first).
- **Cell 19:** the cross-dataset comparison figures `figX1`, `figX2`.
- **Cell 21:** publication figures fig1–fig6 — the per-dataset benchmark bar, metrics heatmap, CDF/hit-rate, RMSE vs MAE, per-seed stability strip plot, anchor mix bar.
- **Cell 23:** figures fig7–fig9 — MARST vs all, MAE vs JamMAE scatter, family-mean comparison.
- **Cell 25:** figures fig10–fig13 — qualitative imputation (you can see actual missing-data fills), missingness map, spatial interpretability, cross-dataset summary.
- **Cell 27:** the sparsity sensitivity curve PNG.
- **Cell 29:** recompute metrics from saved predictions (no retraining).
- **Cell 31:** dataset summary table for the paper.
- **Cell 33:** multi-dataset MARST anchor ablation matrix.
- **Cell 35:** Wilcoxon signed-rank test — is MARST's win over the best baseline statistically significant?
- **Cell 37:** parameter counts per model.
- **Cell 39:** multi-rate (sparsity) main results table per dataset.
- **Cell 41:** Hit@N tolerance sensitivity from saved predictions.
- **Cell 43:** Ridge α and KNN k sensitivity tables.
- **Cell 45:** soft-LOCF decay sensitivity table.
- **Cell 47:** curriculum vs fixed-sparsity table.

---

7. What runs when?

When you hit "Run All":

1. **Cells 0–12** define everything but don't process any data.

2. **Cell 13** defines `run_dataset` but doesn't call it yet.
3. **Cell 15** is the one that actually starts work: it calls `run_dataset('PEMS-BAY')`, then `run_dataset('METR-LA')`, etc. Each one trains 13+ models and saves a JSON.
4. **Cells 17–47** read those JSONs and produce the tables/figures.

That's why "cell 13 takes hours" — it's where all 13+ models per dataset get trained.

---

8. The shape of the multi-GPU speedup

The slow part is training MARST and its variants. With concurrent execution:

- Main MARST has 3 seeds → on 2 GPUs, ⌈3/2⌉ = 2 sequential slots.
- Anchor ablation has 7 variants → ⌈7/2⌉ = 4 slots.
- Decay sweep has 3 variants → 2 slots.
- Curriculum-fixed = 1 variant → 1 slot.

So the MARST family was 14 sequential runs; on 2 GPUs it's ~9 slots. Combined with the halved ablation epoch budget, the wall time is roughly a quarter of the original.

The 13 deep baselines (cells inside `run_dataset` before MARST) are *not* parallelised across GPUs in this notebook — each one trains its 3 seeds sequentially. They're smaller and faster, so the gain wouldn't have been worth the refactor cost.

---

9. Reading the printed output

When you watch the run scroll by, here's what each chunk means:

```
################################################################################
#  RUN DATASET: PEMS-BAY  (speed, 80% sparsity)
################################################################################
Loading data for PEMS-BAY ...
  CSV (timeseries): PEMS-BAY.csv
  Detected: T=5000, N=325, kind=speed
  Valid fraction: 1.000
  Clamp range: [0.0, 120.0]
  Regime edges: (60.0, 65.0) mph | Hit tol: (3.0, 6.0) mph
```

Sanity info: dataset is loaded, has 5000 timesteps × 325 sensors, all entries are real, jam threshold is below 60 mph.

```
1. Historical Average (HA)                              MAE: 3.1198 +/- 0.0051
2. LOCF (Last-Obs Carried Forward)                      MAE: 1.9592 +/- 0.0123
...
18. iTransformer (cross-sensor, causal)                 MAE: 1.6767 +/- 0.0098
```

One line per baseline with mean ± std across the 5 eval seeds.

```
Training MARST 3 seed(s) across 2 GPU(s) at 800 epochs each...
  [MARST(seed=1) seed 1 gpu 1] ep  500/800 | loss 0.0533 | pi[LOCF=0.61 HA=0.33 KNN=0.06]
  [MARST(seed=0) seed 0 gpu 0] ep  500/800 | loss 0.0700 | pi[LOCF=0.69 HA=0.27 KNN=0.04]
```

Confirms both GPUs are training simultaneously — seed 0 on GPU 0 and seed 1 on GPU 1 print interleaved.

```
MARST per-seed MAE (mean over eval masks): [1.4405 1.451  1.4549]
MARST MAE: 1.4488 +/- 0.0061 mph  (over 3 training seeds x 5 eval masks)
```

The headline. Three numbers from three training seeds, each averaged over 5 eval masks.

```
>> JamMAE leader matches MAE leader: 'MARST (ours, multi-anchor)'.
======================================================================
LEAK AUDIT - corrupt held-out truth; leak-free => predictions unchanged
======================================================================
  MARST (ours)   held-out-truth leak: max|dpred|=0.00e+00  -> NO LEAK
  ...
AUDIT PASSED: no model reads held-out data; ST model is strictly causal.
```

Sanity guarantees: nobody cheated.

```
Saved results_PEMS-BAY.json
```

Done with this dataset; results are persisted for the analysis cells.

---

10. If you want to change something

A few common knob-turns:

- **Train faster (lower quality):** drop `TRAIN_EPOCHS` and `TRAIN_EPOCHS_MARST` to 200, drop `ABLATION_EPOCHS` to 100.
- **Run only one dataset:** change `DATASETS_TO_RUN` in cell 15.
- **Skip the ablations:** comment out the cells inside `run_dataset` that build `_ab_jobs` and `_decay_jobs` (the anchor and decay sweeps).
- **Try a different sparsity:** change `SPARSITY = 0.80` in cell 3 to e.g. `0.70`. Every reference downstream picks it up.
- **Skip multi-GPU:** the code already handles this — on a 1-GPU machine, `_train_parallel` just runs the jobs sequentially.

---

11. Common pitfalls (and what we fixed)

- **Device mismatch.** When you split work across GPUs, every tensor has a "home address" (cuda:0 or cuda:1). If two tensors with different homes try to be multiplied, PyTorch crashes. We fixed two of these in this notebook: 1. The MARST trained on GPU 1 was being left on GPU 1 after training (commit

`2a92d9c`). 2. The Fig 5 scatter assumed every model's per-seed MAE array was 1D, but MARST's was 2D (commit `6b111cc`).

- **Leakage.** Forgetting to zero the adjacency diagonal would let a sensor be its own neighbour. Forgetting the train/eval gap would let LOCF carry training values into eval. Both are explicitly guarded against.
- **Race conditions.** Concurrent threads using the same global random seed step on each other. We use per-thread `np.random.default_rng` and per-device `torch.Generator` to avoid this.

---

12. Summary

You now know:

- **What problem** the notebook solves (impute missing sensor readings, 80% blind).
- **What MARST is** (a transformer that blends three cheap anchors and adds a small correction).
- **How it's trained** (3 seeds, 800 epochs, jam-weighted Huber loss, 60→80% sparsity curriculum).
- **What's compared against it** (18 baselines, from HA/LOCF up to iTransformer/BRITS).
- **How it's evaluated** (5 eval masks, MAE + JamMAE + RMSE + 7 other metrics, leak-audited).
- **How the multi-GPU plumbing works** (concurrent per-GPU jobs via `ThreadPoolExecutor`).
- **Where every piece of the code lives** (cell 3 = setup, cell 13 = the giant pipeline, cells 17–47 = analysis).

If you opened the notebook now, every section header should make sense, every print statement should land somewhere recognisable, and you should be able to predict what each cell will do before you run it.